

Hi, my name is Chad and this is your zero to hero Kubernetes fundamentals course. Now, as your instructor, you should know I am CKA and CKAD certified. I've also been an education specialist for 20 years, which means that I have spent the majority of that time in actual classrooms with actual students, not just from talking behind a camera. which means the content of this video has been tested, bloodied, and proven to work for many folks that are looking to get started in Kubernetes. So, the plan

in this video is that we are going to be going through all of the fundamentals of Kubernetes, everything that you need as a foundation to pass your CCAD or your CKA exam, or if you're just looking to get proficient with Kubernetes in general. Now, batter up, cuz here's the pitch. This video is a great starting point, but if you are looking for hands-on labs, AI support, human support from folks like me, then get yourself over to Alta3 Academy, where we offer full courses, which will cover

everything you need to know about Kubernetes, including mock exams and everything that's needed for the CKA and the CKD exams. That link is inside of the description, and I would love to see you there. Here is a list of all the content that we're going to be talking about inside of this video. If you are already somewhat proficient with Kubernetes, use the timestamps in the description to be able to hop to whatever it is that you need to learn. However, if you are new to Kubernetes, I strongly suggest that you just go

through the content in the order that it is presented. You ready to rock? Let's get started. To begin, before we start getting into the what is Kubernetes and the how do I use Kubernetes, let's talk about the why even use Kubernetes. Kubernetes is all about enabling a certain sort of lifestyle in application deployment. Consider this that ultimately it comes down to we've got some code along with dependencies that are needed in order for that code and its application to run. in a style of

application deployment known as microservices. That typically means, hey, let's put all of that code and its dependencies inside of an isolated environment called a container. Now, that comes with a whole bunch of advantages. It's really easy to spin up containers, and if you have any experience with Docker, that's great. If you don't, that's not a problem at all. It's really easy to replace containers, which means that if you need to update or make changes to these applications,

that can be done in a snap. And then there's the whole joke, oh, it worked on my computer, then we'll ship your computer. That whole joke is pretty accurate into what containers do. That if you create it so it works inside of one container, then it should work when you deploy it elsewhere. And a big big part about what's cool about containers is that they are scalable. You can create as many different copies of them as you want. Thing about containers is they are not meant to hang around

forever. Whether they die because you tear them down or because something happens to them. I mean, that's sort of an inevitability. And while there certainly are manual ways to deal with things like that, here's where we start getting into some more problems when it comes to container management in the traditional sense without Kubernetes. So a container in order for it to work has to run on top of an existing environment. So let's say a virtual machine. And you can create as many containers inside of this virtual

machine as you want so long as resources permit. But eventually you're going to get to the point where

oh I don't have any more room for containers. Your virtual machine is tapped. But you need more. You still need to scale out. Well, let's talk about how that could lead to a couple of different disaster scenarios. Scenario number one, your host machine runs out of resources. You don't have any more room for containers and as a result, containers may like not be able to expand. Issue number two,

containers die and they get ignored. There are ways for you to work around this, but you have to work fairly hard. And spoiler alert, this is something that Kubernetes can handle for us. And the third scenario is that all these containers are running on the same virtual machine, right? So if your virtual machine dies, no, all those containers just went down with it and there's nothing that's set up in order to bring those back. Fortunately, Kubernetes has an answer. Here's where we might be taking your

prior knowledge about Docker and applying it to how it works inside of Kubernetes. Containers and Kubernetes work exactly the same way they always did, whether they're being created by Docker or something else. But Kubernetes containers are placed inside of Kubernetes objects called pods. Pods are really just wrappers that are being placed around containers. Kubernetes manages pods, not containers. So by putting them together as one unit, it makes it easier to control. So that package deal, that pod and container

still needs to live on top of some environment. The container needs to draw its resources like we saw before. Traditionally, that can be on a virtual machine. In a Kubernetes environment, when you add virtual machines to it, those are called nodes. And you can add multiple nodes to the same environment. When you do that, you wind up literally exponentially increasing the total amount of resources that might be used for lots of pods. And when you combine all of those nodes together, you get what are called a cluster. Now,

there are two flavors of nodes. The ones in green on my screen right here, these would be our worker nodes. And this is where you would expect to see the vast majority of pods, which remember are wrappers around containers to be created. And as you can see, there can be more than one pod on like a given node. There can be uh a disproportionate number of pods on these different nodes. Some of these nodes may be in different sizes, like the one on the right is smaller, therefore maybe it can only fit

one pod. Whereas the one on the left is bigger and that one can hold three. A cluster is just an aggregate of resources. Now the other type of node which is again also a virtual machine is known as the controller. The controller does exactly what it sounds like. It is the hub of all of the tools that are required in order to keep your cluster running. And we're going to talk more about those later. As a general rule, you don't want to be just willy-nilly putting pods on top of the controller

node. Why? Well, because the controller node's busy. It needs its resources to, you know, manage the cluster, which is pretty important. Now, before we get to the fun stuff and before we actually start building pods and things like that, it is important to have an understanding of what is going on under the hood. what's ticking and running inside of these nodes and what tools are we using in order to access and interface with our Kubernetes cluster. The first tool that we need to learn and

one of the many things that starts with the word kube because Kubernetes just loves putting kube in front of everything is kube control. And kube control is the CLI tool that is going to be installed inside of

your personal workstation, your laptop, wherever it is that you're working from. That is where kube control is installed. And what that enables you to do is that from your machine, you are able to access the cluster. Note that we're doing so by connecting to the controller node that

we spoke of earlier and that all kube control requests are going to be received by the API server. So all communication inside of Kubernetes is done through an API which if you know about API APIs and HTTP and all that stuff great. If not, it's not a deal breaker, but we'll talk about why that's going to be relevant later. Now, in order for kube control to talk to a given cluster, after all, what's stopping somebody from installing kube control and talking to any cluster in the world is that it must be configured

to do so. So, if we take a look, what you see here in miniature is a kube config file. This is a document that should be on the same machine where kube control is installed and it includes such things as like the location of the cluster, but also includes where is the certificate and key file again located on your personal environment that will allow you to authenticate who you are and talk to that cluster. Without that setup, kube control is never going to work. But moving on through the anatomy

here, talking a little bit more about the API server. Like I said, all communication like all action that occurs inside of a cluster is done through an API, which means that a request is sent to the API server. It decides whether or not that request is valid and then as a response will take certain action or return certain information depending on what it is that you're trying to do. So the API server is what will be returning information to you here at kube control. It is what also reaches out and communicates to

every single node that are part of our cluster. Remember, you can have multiple nodes, multiple virtual machines, all part of the same cluster. The API server is also the only thing that talks to ETCD, which is what we'll talk about next. All right, we're here on the official Kubernetes website, which means that this is definitely third-party and it's not just something that got invented by Kubernetes. ETCD was around long before Kubernetes was. And what is ETCD? It is a strongly consistent distributed key

value store that provides a reliable way to store data that needs accessed by a distributed system or a cluster of machines. Well, you just described the cluster there, right? Distributed system of cluster of machines. So, stores state. It's all about storing data. So, specifically when it comes to Kubernetes, where is it being written? How many nodes are attached to a cluster? Where is it written how many pods are running? And are they actually running? Are they erroring out? Like where is it written that whether or not

pods are able to talk to each other and things like that. Just Kubernetes did. And as a result, ETCD is something that as either an administrator or as a developer when it comes to Kubernetes, you should be very respectful of it because losing it is like labotomizing your cluster. You are going to lose all track of all things that are running inside of it. Another component that is part of the controller is the scheduler. And what this is doing is that it's put this pod up on this node right here. No. Okay. Like Tetris

theme. What happens to pods is that they get put on nodes, right? Well, what makes that decision? Sure, we can make that decision ourselves, but as a result, we want Kubernetes to decide that. So the scheduler is what's going to try to make sure that you've got like an equal distribution across your

nodes for the most part plus obeying any sort of rules that get put in place either by you or by the system itself just to make sure that everything is following the rules. Speaking of following the rules,

there is the controller manager. You might picture the controller manager as like the sheriff. I'm here to make sure all you pilled stay in line. I'm here to kick butt and chew bubble gum. And I'm all out of bubble gum. The controller manager actually manages a lot of different daemons that control a lot of the behaviors of how your cluster runs. Now, if you're brand new to Kubernetes, these may not mean a whole lot to you right now. But for instance, objects in Kubernetes are tracked via namespaces

that they can be controlled through role-based access control which are connected to service accounts that you'll be creating copies of pods called replicas. That's something that needs to be tracked as well. And the controller manager is what's in charge of all of that. Now, Culet, Culet is present on every single node inside of your cluster. And regardless of what this diagram might be showing you, Culet is also present on top of the controller nodes as well. But Culet is kind of

like the eyes and the ears of every node inside of your cluster. It's responsible for all sorts of things. making sure that the containers are started, stopped or restarted appropriately on each node in your cluster. Any instruction that comes from the controller node is going to be received by Culet. So suffice it to say, if Culet encounters problems on one of your nodes, well then your node is now in trouble because that is your sole connection to the rest of the cluster. And now what about Docker? A

lot of people say, "Well, should I be using Docker or should I be using Kubernetes? And if not Docker, some other container runtime engine." Here's the thing. It's not either or. If you're using Kubernetes, you are using Docker or containerd or whatever. Because here's what's happening. All those pods that are being created on all those nodes inside of your cluster, remember inside of those pods are containers. just exactly as we've always understood them. What's building those containers?

Is it Kubernetes? No, absolutely not. Because installed on every single node inside of your cluster is going to be some container runtime engine, whether it's Docker or something else. Which means if you've ever used Docker and you've used such commands as `docker run` or `podman run`, which creates containers, guess what? Those commands are still running in Kubernetes, but who's writing them? Not you. Culet. So Culet, our boy that we were talking about earlier, is what is responsible

for all of these Docker or whatever commands that start, stop, restart, gather information about, but they are still using that container runtime engine. And what that means is that the life cycle of containers as we understand them still absolutely apply inside of Kubernetes. You still need a image which is to say like a picture a template for how a container should be built and the application inside of it should be set up and images still need to come from somewhere from some place of storage. Those are called registries

and that of course can be configured to all sorts of different places. Now let's put all of those moving parts together and let's see how does it wind up that if we say make a pod that those now let's bring all that together and let's see what would happen when we actually use `kubectl` to make a pod inside of our cluster. Step number one and of course we'll be doing this ourselves like real soon. We use `kubectl`

control commands to create a pod. We'll talk later about how you're going to

write a document called a manifest and that will include all of these instructions on what Kubernetes is supposed to do and how it's supposed to build it. So that information is taken to the controller in this east west diagram. You can see that right here which of course is received by the API server. Now you can see just how much traffic goes to this API server. It's quite popular. The API server will then turn to Etsy and assuming it doesn't just reject your pod plan outright because there might be typos or you

might not have authorization yada yada. If it passes all that then it will tell hey here's the pod we're going to make. Etsy sends back a response. 200 in HTTP speak means thumbs up. Sounds good. API server then sends a response back to you at your command line saying, "Yeah, looks good, boss. We're going to go ahead and make that pod. But is the pod made yet?" No, not by a long shot. What happens next is that our API server needs to know where is this pod going to go? So, it goes to

theuler. And assuming that our manifest did not say explicitly where we wanted the pod to go, theuler is what is going to make that decision. Let's go ahead and put that on top of worker node number two. So just remember worker node two is the decision that got made that sent back to the API server. API versus server says, "Okay, sounds good. I'm going to let ETCD know that we're going to put that pod on node two." CD says, "Sounds good, boss." And then API server

says to the scheduler, "Hey, good pick. We're going to wind up going with node two." So it's a lot of back and forth. All right. as we are ascertaining where resources are going to be located physically inside of our cluster. From there, our API server, which again is inside of our controller node, is going to reach out to the worker node where we decided we wanted to make the pod in the first place. Remember how we said that that might have been worker node number two. So the API server sends that to Cublet,

specifically Cublet on worker node number two. Cublet says, "Got it, boss." And then turns around and runs all of the necessary commands like docker run or container dart like whatever the command is. It's cublet that's handling that for us. And as a result, it is only at that moment that a container is actually built. And it's also cublet that's responsible for making the pod that the container was put in. So it's not a question of which came first, chicken or the egg. It's definitely

clear pod is made first, container is made inside of it second. And what Cublet will constantly do is that it will be feeding status updates as the container is being built, as the container is finished, as the container enters an error or stop state. Cublet is perpetually providing that information back to the API server, which then will in turn make sure that that information is being recorded to ETC. So, it's a really nifty little ecosystem of what's happening here. And having an understanding of what these components

are and how they work together can really help you troubleshoot and better understand how to make the resources and make sure that they are working and healthy inside of your cluster. So, very shortly we're going to be creating objects. What you're seeing here is an example of how to do that by writing a file called a manifest. So, a manifest like making a pod like you're seeing here just provides a list of descriptions of these are all the things that I want. So, it pretty much is the

same concept as you're going to your local coffee shop and you're punching into your app like make sure that it's oat milk and that oh there's this this and this topping and oh, leave room for I don't know. But point is that's really all that a manifest is just writing out this is what I want and this how I want it put together. Now the problem is is that manifests are written in the YAML. If you don't know how YAML is written or edited, let's talk about that. If you

already know about YAML, feel free to skip this section. All right. When you're working with YAML, there are two different types of data that you need to be aware of. The first is definitely the easier. This is referring to lists or sequences. People can call it by different names. So imagine that what you're seeing right there is my grocery list. It's literally a bunch of things that I need to get. It's separated by commas. Fine. But in YAML, lists look like this where every item has one of

those dashes in front of it. So it makes it far more visible and readable. And that's really what YAML is all about, making it readable to human eyeballs. The other type of data we need to know about are dictionaries, also called mappings. And these are very different from the way that lists work in that dictionaries are all about key value pairs. It doesn't matter what order these key value pairs are in unlike a list. But what you're seeing there in blue are all my keys. What I'm going to

put inside of white right here, let's say chocolate flavor. That is the value to the key flavor. Like what brand of ice cream do I want to have? Hershey's or you know when I'm buying something in the store I don't want it to like expire any sooner than like June 30th for instance. So notice that there is that colon that separates the key from the value. And you can have key value pairs that are for instance booleans. Like here I've got true. And if I'm trying to say calories like about a,000 calories

for a carton seems a small price to pay. You can have all sorts of different values like that. Okay. So now we can see what lists and dictionaries look like inside of YAML. But here's the thing, they compound. They get more difficult. They get nested. A lot of stuff starts happening. Let me show you a few more things. So flavor, brand, expiration, and all this other stuff. These are all describing my ice cream, right? So what I might choose to do is I would say that ice cream is the key. The

value of ice cream is all this stuff. Like that's what's describing it, right? Like how am I to say that everything I have highlighted is the value of ice cream. Think by hitting it with an indentation. Indentation is critically important in YAML. It implies ownership. And I'm in Notepad++ here. Notice that there's this little button that I can use to collapse. This makes it very visible in showing you that, hey, all of this data is the value of this key. And where things start to really get fun is

when you have nested values inside of others. So, under ice cream, I'm writing out my flavor and brand of ice cream. But maybe I would like to include what toppings I want on top of that ice cream. Here's the thing. Maybe I want a list of toppings. Maybe I want sprinkles. Maybe I want some fudge. Maybe I want a cherry and some whipped cream on top. Notice how that list of dashes is indented under toppings. Toppings is just another one of those keys that's indented under ice cream. When you look at manifests, they

all follow these rules. And any indentation or dashes that you see are there for a reason. And we'll talk about why that is when we explore some of these other manifests in a little bit. All right. So, let's talk

about making manifests. Certainly, at first, what you're going to want to do is you want to get your manifests straight from the source. Never ever try to write them from scratch. What I got here is the Kubernetes web page, and you can search for whatever it is that you're trying to

make. So let's say pods as our first choice. And when you often go with the very first result, just scroll down and you will find that there are all sorts of different manifests for all different ways that you can use certain resources as well as explanations of like why and how they work. So the first manifest we're going to look at appropriately is a pod. And the good news about manifests is that even the longer and scarier ones, they all work pretty much exactly the same. And once you learn how one

manifest starts to work, a lot of the other ones start making sense as well. What you're going to see inside of all manifests are these three values at the top. API version, kind, and then metadata. Kind, fairly self-explanatory. It tells us what kind of object we're working with. Be careful though because these are case sensitive. If you put in a lowercase P for pod, oh that's just going to cause a big big problem. So it's little things like that that will bite you and you'll learn very quickly.

Metadata is fairly straightforward. Again, notice the indentation that we talked about in YAML. Anything indented under metadata belongs to the metadata section. So label later when we're working with things like labels and annotations, those will go under the metadata section as well. Now API version is kind of interesting. You might recall from when we were talking about the architecture of a cluster earlier that everything in Kubernetes uh is communicated through APIs. Well, there isn't just the one API inside of a

Kubernetes cluster. There are many. And all of these different APIs are configured to configure and work with different types of objects. Version one, which is like the most primitive API there is. It's been around the longest, is the one that is proficient at creating pots. So, it is super important that you get the API version value here correct or you're going to get some sort of error where it says API version D cannot support whatever type of object you're trying to make. And then that

brings us down to spec, which of course is short for specification. This is where you get to put in your order, so to speak, of this is how I want my object to be built. Now remember a pod is just a wrapper around a container or containers. You can put in more than one. Now look under line six where the very first part right there is a dash. Remember dashes indicate elements inside of a list which means d I could actually put in a list of containers. If I wanted more than one then I would basically just do something

like this. This is engine x2 and this is using something else xyz. So that's the significance of that dash there which will be the significance of anything including like ports like hey what ports are open inside of that container. Let's make a list of all those. Now of all of the fields that we're placing inside of a pod manifest where you certainly want to be the most careful is right here. Your image that name needs to be accurate. If there is any sort of typo like I did just there, you're going to

have a bad time because it's not going to find that image. It's going to try to find engine X and it's going to cause errors. That happens to everybody, but forewarned is foremed. Okay, so in a moment we're going to start building our first pods here. And before we can do that, there are a bunch of really fundamental kube control commands that we need to know. kube control get is the most basic there is.

It returns a list of whatever object it is that you're asking for. If I type

cube control get pod, I'm getting back this no resources found. That doesn't mean that there's no pods running inside of my cluster at all. Notice that what it's saying is no resources found in default namespace. We're going to talk more about namespaces later, but suffice it to say in our current namespace, no other pods exist. So, we need to change that. So, what I'm going to do is I'm going to create a manifest here. Pod manifest.yaml and we're going to paste in that manifest that we were looking at

earlier. All right, nothing new here. Save and quit. So, what we're now going to do is we are going to create that pod. Now that we've got that manifest created, now there are two different commands you could do here. Cube control create or cube control apply. Cube control apply is the one that you see more often. But basically the way that it works is that I would like to apply what is at this file location -f pod manifest.yaml. And what this is going to do is that if that pod doesn't exist, it

doesn't cube control apply will create it. We are literally applying what is in the manifest against the cluster. That's what makes Kubernetes declarative, which is nice. When we pull the trigger on that, we got this message. Cube control uh whatever got created. Let's do a cube control get pods. All right. Statuses container creating. And if we just give that a second, eventually it should say running. But while we're looking at cube control get, let's take a look at what we're seeing here. this column ready

01. If you remember from our manifest, how many containers did we put inside of this pot? Just one, right? It was running engine X. And how many of those containers are actually in a ready state? They are up and running. Well, zero. This one's still being created, right? So, if I run that cube control get command again, now you can see it's in a running state. It's one out of one ready. Also, there are zero restarts. When you see a lot of restarts on a pod, that's a big red flag. It means that

something is causing that pod to die and it's something that probably needs to get fixed. All right, so now that we've created our pod and we can see our pod, let's talk about our next command. cube control describe. Now cube control describe does exactly what it sounds like. It will describe information. And unlike get which just returns a simple list, this is going to give us much more information. I'm going to describe that pod that I just made called engine X. And take a look at the output that I

get. That's a lot. Cube control describe is great for double-checking if resources have been created the way that you anticipated them. What's also nice is that at the bottom of a cube control describe is this section here called events. And if a resource is busted, if it's not working, describe should be the first tool that you reach for because you'll often see somewhere inside of here what you may have goofed up and caused it to not work. And now for our last command, delete. All good things

must come to an end here. And let's say that this pod needs to come down. There are several ways that you can delete resources, but here are two of the easiest. When we think of the phrase I made you, I could destroy you. Remember how we did apply -f with our pod manifest? We could use cube control delete with that file and it will delete that for us just the same as it created it for us. That's option number one. Option number two, if you don't have the manifest or you don't want to go looking for the



manifest, you

can always just cube controldee. Specify what type of object you're deleting. a pod and then what's the name of the pod you're trying to delete? Engine X, right? Either way, the result is the same. The pod will be deleted and after I check to see, we can see that that got removed and we've created, inspected, and deleted a pod at this point. Next, let's talk about namespaces, which was coming up a little earlier. Right? If I were to type cube control get pods right now, it says that there are no pods

found in the default namespace, whatever that is. But if I use my cube control get command this time against namespace, you can see oh default as it turns out is literally the name of the namespace, not like the default namespace which happens to be XYZ, right? There are actually many pods running inside of my cluster right now, but I can't see them because I'm not looking in the right place. So check this out. When I type cube control get pods, but this time I specify -ashn for namespace. And let's

take a look at cube system. Now I'm seeing some pods. Now these pods that are running inside a cube system are system integral pods. They are what's holding our cluster together. And do you remember when we were talking about some of like the important components that run inside of Kubernetes like CD and theuler and things like that? Depending on how your cluster is built, you may actually see pods that are running those applications in pods inside of this namespace. Our training environment is set up a little

bit differently. We can talk more about that inside of the real course. But point is is that these pods are here because that way they are not getting underfoot when I'm trying to create business related pods. And also nobody should be looking at these pods except for admin anyway which access control is a big big reason for wanting to use namespaces. So that being said, it is not difficult to make new namespaces and I'm going to do this imperatively, which means I'm going to make it at the

command line. Not going to use a manifest at all. If you're planning on taking the exams, using imperative commands like this are a super quick and easy way to generate certain objects. It's what I'm going to use for my demo. In fact, let's create this namespace and let's just call it demo. And what I'm going to do is I'm going to hop back inside of that pod manifest that we were looking at earlier. Now the thing about manifests is is that when you are creating objects, if you don't specify

where that object is supposed to go, it will go to a default location. That's set by a context, but that's a conversation for later. Something that you can do in order to force pods or other objects to go into certain namespaces is to just add under metadata namespace demo. If I were to create this object now pod manifest.yaml says that the pod was created. If I try to see what pods are currently running, I still get no resources found. Did I screw up? No, of course I didn't because those pods are

inside of the demo namespace. So, think for a minute just what an easy mistake that can be making objects inside of incorrect name spaces and the confusion that that can cause. Happens to everybody, but forewarned is forearmed. Now, consider this a preview of when we start getting into resource control a little later. But remember, Kubernetes is a shared environment and you need to make sure that when resources are getting created, you aren't taking up more than your fair share or you're preventing pods from

going hog wild and just consuming too many resources. Well, what I'm about to make here is a resource quota. And a resource quota is something that attaches to namespaces and sets the rules. Hey, look. Here's the hard limit of what everything inside of your namespace is allowed to consume. CPU one core, memory, one gig. And what I'm also going to use this as an opportunity to show you is is that there's another way to specify where resources get built. Resource quotas attached to namespaces. So instead of

writing that under metadata, I'm just going to type dash N demo. So when I hit enter on this, you can now see that my resource quota has been applied. And again, I'm using kubecontrol describe to be able to confirm that. And any pods that get created in this namespace are now going to have to adhere to those resource consumption rules. That's something really cool and something we're going to talk about a little bit later, but again gives you some more insight into how important name spaces

are. Let's swing back and let's talk about that API version line which is inside of all manifests again. Now, as you can see, I wrote something entirely ridiculous right there. There is no API version honkabonka in Kubernetes. At least not yet. But point is is that I want you to imagine that this is the scenario. The day is yesterday. This manifest worked. Version honkabanka is good. Each and every one of these different parameters that you set were all accurate. Things were doing well.

Today we rolled out a new cluster at a later version than the one we had yesterday. Suddenly your manifests don't work. And suddenly you're getting messages that look like this. There are no matches for kind pod in version honkabona which is the hilarious message that I actually want to see. Yeah, there we go. I thought version two was too boring. So what is this saying? Remember Kubernetes communicates everything through APIs. And when you are upgrading Kubernetes, what you are upgrading are

its APIs. you are upgrading its ability to recognize, configure and manage different kinds of resources and those change. So what does that mean for you? Well, if you want to be like a real pro at Kubernetes, like if you just want to make everybody think you're such a pro at it, every time that somebody talks about upgrading a cluster, I just want you to go just like make that exact sound because nobody likes upgrading clusters. It's such a huge pain. And looking at this deprecated API migration guide

here, just take a look with me. In version 1.32, it's telling us that this type of resource called a flow control, it's API version went from v1 beta 3 to v1. What it's also telling us is that and it's using dot notation here. Remember how in pods like under spec we would set containers and then the name of the container. Here it's saying that for this particular object under spec is a section called limited and under that is a key called nominal concurrency shares. Right? It's telling us that its

default values are changing. All right. So that's what happens as you go from version to version of Kubernetes. You have to update your API version. Suddenly fields might mean something different than what they originally did or some fields might get added or some fields might get taken away. And you can scroll through this and you can see all of the different changes and that's just the nature of the beast when you're working with Kubernetes. Let's talk about resource management next. So a

Kubernetes cluster is an aggregate of all the nodes that are worked together in order to create one whole. And what you have to be doing is constantly keeping an eye on how many resources are being consumed and setting up reasonable guideposts and guard rails to make sure that isn't being exceeded. There are lots of third-party tools that you can integrate into Kubernetes. Like some of them really pretty and really visual, but there are a few that you can uh install that are easily used, even the

most vanilla of setups. Cube control top is one of them, but cube control top, if you are building a cluster from scratch, chances are it's not going to work. You're going to get this error metrics API not available. What we need to do is that we need to apply this uh manifest here which is provided by a GitHub repo from Kubernetes. And what that's doing is making a whole bunch of objects. Some of which we'll talk about inside of this course, some of which we won't, although

we will of course inside of the full course. Um, we need to give that a minute to spin up. But what I just did is that I just created a bunch of pods that are gathering and harvesting information about other pods in the nodes inside of our cluster. These uh cluster roles and roll bindings and cluster roll bindings are what are giving those pods permission to do that. And the stuff about service accounts and stuff can wait uh for a little later. Point is is that we want that to run for a little bit. And I'll just kind of

pause here for a second and let it finish up. There we go. Once that API service uh metrics is telling us that, hey, I'm up and running. True. Now I can start using cube control top. So here's that cube control top nodes command again. Check it out. It's telling me what percentage of available CPU and memory has been used on the nodes inside of my cluster. That's really nice. What's an also a nice use case for cube control top is pods. If I were to cube control top pods, remember just like with cube

control get and describe, it's only going to look for pods in the current namespace. I don't have any pods in my current namespace. So what I can do is I can type top pods all name spaces or if you want to look cool dash capital a and here we can see that uh the pigs of our uh cluster are definitely these two calico pots. That's to be expected. They are our networking uh what's holding our nodes together basically. But this is a great tool to like quickly see like which of your pods

are like just out of control consuming the most resources which is a great way to troubleshoot and figure out what needs to be done to rectify the situation. All right, a little death by PowerPoint here, but we're going to continue talking about resource control. What we're going to be doing really soon is that we are going to be editing our pod manifests and we're going to give them more parameters and we're going to make them more in line with the way that we want them to behave. Specifically, we

want them to behave themselves when it comes to resource consumption. And it's a fine line to walk, right? Because the containers inside of your pods, they need resources to live. You don't want them to starve to death. But at the other hand, you don't want them to be pigs. You don't want them to consume more than they need. So there are ways to set those. Requests are a parameter that you can set inside of your manifest that guarantees that your pod has a certain amount of resources. That's good

because otherwise your containers are going to starve. Give them the give them the oxygen, man. They need to breathe. And then there are limits. It's not unheard of for containers to just go a muck and just start consuming like rogue like amounts of resources. Limits are what put a hard cap on how many

resources containers can shove inside of their face. I think that's much easier to understand because I'm pretty sure we can all uh identify. All right, so what would it look like adding these on to our

manifest? Let's take a look. Now it's important to note that when we are controlling the amount of resource consumption here, look at the level of indentation. Do you see how we are adding this section to this particular container? So requests and limits are set on a container by container basis. Make sure that that's clear. And under resources is where we can add those two sections, requests and limits. And indented under that is where you can control. All right, what's the minimum

amount of resources as far as maybe CPU and memory is concerned? I want my pods to have at least let's say 65 uh megabytes of memory. And when it comes to CPU, here's something unique to Kubernetes. If one core is equal to 1,000 milliores, the heck is that? Millor allows you to like further specify there's only so much of the CPU timing that I want to use. I don't want to use a full core. I can actually say at the very least make sure that I get 250 mil core worth of CPU. And pretty much the same thing

applies down here at the bottom, except limits must be at least the same as the requests. I would get an error if I tried to set the limit lower. But let's set a limit of 500 and let's set a memory of 130. Basically doubling it. And if I were to create this object right now, it would work. Now let's take a look back at that name space we were looking at earlier. Do you remember when I put a resource quota on top of it saying that uh hey any uh pods that get created inside of this namespace are going to have to follow

the rules. Well, watch this. If I put a watch on here, then we can see this happen in real time. But I'm going to go ahead and create this pod here. And I'm going to put it inside of the demo namespace. The second that I did that, look what happened. Immediately that resource quota said, "All right, you've already consumed x amount of resources. You've already requested that much to be used." And what that means is is that I could keep making more pods exactly like this

one, but only up until I reach what the resource quota amount has been. So a resource quota, the way that it works is that it's kind of like the bouncer that's standing outside of the nightclub. All right. When you've hit the maximum amount of people that you're allowed to let inside of the club because you know like there's fire code rules or something, at that point you start turning people away. you start saying no, no more, we're full up, go home. It's the same with making pods.

Let's just do an illustration real quick to see how that changes. So suppose that we create a pod with that request and limit in a certain name space. We have a resource quota where it says that hey you can only use up to 400 megabytes in memory, 500 megabytes in limits. Now, what's interesting about this is is that what the resource quota now is doing is that it's being much more granular. Resource control can be super granular to the point that it's kind of confusing, but it is possible to really

spell out in your resource quota just exactly how high you want certain amounts to go. So watch what happens when we add another pod that also has a request of 100 and a limit of 200. Notice that with that limit of another 200, we are currently up to 400 megabytes of the 500 that we're total allowed to have here. If I try to make one more pod, that is going to be in excess of 500, which means that this pod would be denied. it would not be allowed to be created inside of our cluster. Remember

that all Kubernetes is ultimately in the service of maintaining containers. And what we're going to talk about next are probes. Now probes are basically watchdog dogs that you can put on top of your individual pods and hence your individual containers to basically look out for and enforce certain behavior. The two types of probes that everybody should know about are readiness and liveness probes. Now, the way that both of these probes work is that you start with a container. And what's happening once you

place a probe on it is that that probe represented by this ghost inside of this sheet is constantly poking that container with a stick. The reason why it's doing that is because every couple of seconds or so it's going to ask, "Hey, you okay?" And then the container is going to reply back, "OMG, yes." And this is going to happen over and over again forever. That's the way that probes work. But the thing is is that there's two different kinds of probes. Like I said, and here's

the difference between the two. If you are working with a liveness probe, when the container is probed by the probe and you get no response back at all or the response comes too late, what the probe will do is say, "Well, that's a strike." And after a predetermined number of consecutive strikes, the probe is going to kill the container. Ironically enough, that's what a liveness probe is for. Because turning it off and turning it on again is just one of the timehonored techniques of it

troubleshooting. Sometimes restarting the container is exactly what you need. Now, a readiness probe is different. It also is checking to see if the container is responsive. But here's what happens with a readiness probe. It doesn't kill a nonresponsive container. This is the gentle parent. It's like, "Oh, little container Timmy isn't awake and ready to face the day yet. Everybody, everybody, leave little Timmy the container alone." Right? He's not ready. I don't want any traffic to go to

him. And what's happening is is that we are basically removing that pod from any predetermined settings we have that allow other pods or other resources to communicate with it. So putting readiness probes on things make sure they are ready before they are actually being accessed. Okay. So what do these probes look like inside of manifests? Well, here's a pod that is running a application simple service. It's a testing tool and we are adding a liveness probe which again notice is being added at the container level. It's

not something you add to a pod. It's something that you add to a container inside of a pod. And I got a lot of different settings here. And this is where working with probes can get tricky. What do these mean? Probes are extremely granular. You get a lot of control over how you want these things to act. Let's start down here on lines 17, 18, and 19. First thing that we need to determine is what is the method of the probe like how is like what's the stick that the probe is using to tap the

container. Well, the simple service uh application here is an API which means it can actually receive HTTP requests. So the way that we're testing that we're probing is that we are sending get requests to a particular endpoint at a particular port inside of the container. Not all containers are going to handle this particular method of probing. There are lots of different things that you can do, however. So we'll just leave it at that. But what do these other parameters up here mean?

Well, initial delay seconds too means how soon after creation are we probing like how many seconds are we going to let it get on its feet before we go tap tap you okay two seconds period seconds this is asking

how often thereafter are we probing so basically what's happening is is that the pod is created. Then two seconds goes by and then there is a probe. Then another five seconds goes by and then there is a probe and then after another 5 seconds well you get that that just repeats forever.

So the uh delay in probes can be significant because if you're probing all the time then that makes for more highly likelihood that you're going to wind up finding an error. Speaking of errors, timeout seconds. How long we giving the container to respond? So after I tap it, tap tap. You okay? All right. Here I'm saying you've got one second to send me back a response to my HTTP get in this example that you are okay and failure threshold. What this is saying is is how many consecutive and

that is important consecutive failures before we kill this not pod container because that is ironically what liveness probes do. They kill containers. All right. And uh obviously you can tweak these like using all sorts of different parameters and hopefully that's making it clear. Make sure you're reading the documentation when you are creating probes. And what's super nice is is that the difference between writing a liveness probe and a readiness probe is virtually nil. I could leave this exactly as is

written. But now the behavior is different. What would happen is is that after I get three consecutive failures to get a response from this pod, it would turn off traffic. It would turn off traffic to that pod until the probe was satisfied after more probes that okay, now it's, you know, answering and now I can allow traffic to go back to it. So, make sure you know the difference between readiness and liveness. What I'm about to do is an exercise in futility. It's to teach you that, hey, what I'm doing is great in

testing, but is not something you'd want to do in production. And it's also going to introduce a really important concept we're going to be getting into next. Plus, I'm going to show you some cool new kube control commands. Kube control run is a quick and dirty way to create pods without using a manifest. So, this is going to make a pod called demo pod. It's going to use the engine X image inside of it. And a neat command that I can use is port forward. If I use port forward against a pod, which is named

demo pod, I can choose a local port, one that's on my local machine to connect to the container port inside of the engine X container itself. Now the container port that's written in stone. Engine X its containers are exposed for HTTP on port 80. For local port, it could be any TCP port that's available on your system. Uh I'm going to use port 2224. Now when I hit enter with that command, it is going to hog up my command line. Now I have T-Mux, so I'll just split my screen. And by the way,

for the exam, T-Mux is installed. So you will be able to use tricks like this if you are studying this to plan to take the exam. With that port forward activated, look what I can do. I can curl localhost port 2224. What's happening here is is that I'm going to the local port that's connected to the container port that's inside of the pod. How cool is this? Welcome to Engine X. This is the HTML. This is literally the welcome page for what's running inside of this container inside of my cluster. You got to kind of

be a nerd to get excited by that, but heck, guilty. Anyway, press control C and that'll stop the port forward. And that means that if I were to try to curl localhost 2224 again, I'm going to get a failed to connect. Now, point is is that I want to make a configuration change to how this container works. So, there are a couple of different ways you could do this. I'm going to use a kube control exec command. And what I'm going to do is I'm going to open up an interactive terminal

inside of my pod which was called demo pod. And what that terminal is going to be is a bash shell. So I'll just type `sh` like so. And check it out. My command line has changed. I am inside of my engineext container right now. And if I needed to make any sort of, you know, configuration changes or things like that, I could. For instance, let's make a `uh` inside of `var`, let's make a `www` directory. And inside of that, I'm going to create a new custom `uh` index page. So, let's just echo `help`.

I'm stuck in a container and I can't get out. That's what I want to be my little welcome page, right? I'm going to put that inside of `var www` inside of `index.html`. Now, making that change, uh, you'll wonder, well, I didn't use Vim or something like that. Well, where's Vim installed? Not inside of this container. If I try to use Vim or whatever, it's not going to work. So, if you are using CubeControl Exec to open up a shell, just be in mind, you're kind of limited

to the tools that are inside of that container at the moment. But again, remember this is an exercise in futility. All right, I'm going to leave that open because I'm going to be using uh my other command line here to make a change. What I've got is a configuration file here, `engineext.config`. That will change the way that this container works. What I need to do is get it from here to there. So, here's a neat trick. CubeControl copy allows you to take local files or files that are remote that are inside of

the pod and you can copy them in and out. I'm going to go ahead and copy `enginex config`. I'm going to put that inside of demo pod and we're going to put that inside of the `var www` directory. No, excuse me. We're going to put that inside of the `Etsy uh enginex` directory. Uh and that `enginex.config` config file that I'm copying in will replace the one that was already there. So, I can confirm that by just checking, hey, inside of the `Etsy engine X` directory, is that `EngineX config` file

changed? Yeah, all these comments are telling me that that's my custom little file. And what it's going to do is that it's going to use that custom index that I made at `var www`. All right. This is just showing you like the method behind my madness here. This is not anything that's like like super essential to understand uh how Engine X works. But here's what we have to do. I need to in order for that to work uh reload Engine X. That's going to kick in those changes. So if that worked, we

should now be able to do this. I'm going to exit back out and let's run that port forward one more time. Let's do a `curl` at `localhost:port 2224`. And check it out. I have created an index file that's custom. I created a configuration file that points to it. I made both of those changes inside of my container using some pretty neat uh uh cube control commands, except ah ah oh. Oh, excuse me. I seem to have deleted my pot. Oops. Well, I could always just remake it, right? Cube control run `demo pod` and let's use the

image engine X. All right, there we go. Pod's back just like it was. I can reopen my port forward. I could run my `curl` command again. And whoa, what happened? Ah, I lost all my work. None of it persisted. All right, this is why this is a, you know, show and not do situation. Great for testing and when you're figuring this stuff out, but the fact that that didn't work is that it is because containers are stateless. All those changes were made to the old container. The old container is dead.

It's gone. And the changes that were made with it are dead and gone, too. So, how are we supposed to

take ephemeral things like pods and containers that die constantly? And are we able to push configurations to it? Glad you asked because that's the next thing we're going to take a look at. All right, let's do a little animation to help make this point clear. So you saw what happened that I made that engineX config file inside of that container. Problem is is that when that container and pod

go away, it is going to take that file with it. So obviously that's not a solution. We can't have engineX.config inside of the container. Where then? I mean it's going to have to be somewhere else. In Kubernetes there exists a type of object known as a config map. This is a whole other type of resource that you can make using a manifest just like you can make pods. And inside of config maps is where you can put pieces of data like configuration files or environment variables or really any sort of thing

that you think might need to be changed about how future versions of pods may need to be configured. It makes it super easy to make those changes in a way that's disengaged from the container that you are attempting to uh adjust. Now, what we're going to be seeing here is that making config maps are very easy. Attaching config maps to containers uh that's something of a different story. If you already have experience working with Docker and volumes, great because this is going to transfer 100%.

But if not, no worries. Let me explain it to you. The config map, which remember has the config file inside of it, will be mounted as a volume to the pod itself. Once that volume has been mounted to the pod, we then mount the volume to the container. and the volume is mounted to the container at a specific mount point which is going to be some arbitrary directory inside of the container itself. Now what's super magical about this is is that remember you may have many different pods that are all

identical and they all require the same configuration. Well, hey, take that one config map and you can mount that to as many pods as you want. So, configuration changes only need to be made in just one place. That's super advantageous. You're going to be seeing config maps an awful lot. Now, let's see what it would take in order to write a manifest that would actually pull this off. All right, step number one. Let's make a config map. As you can see, there's already one config

map right here that we don't need to worry about. and we're going to cheat a little bit. We're going to use a `cubecontrol create` command in order to make it. First, I need to make a file. Let's create a txt file. `Heroes.txt`. Let's put a couple of my favorite heroes inside of here. All right. Superman, Batman, Wonder Woman, Flash. Yep, looks good. If I were to do a `cube control create` config map or again if you want to look like a pro use the shorthand `cm`. I'm gonna create a config map that

I'm going to call `dem heroes`. Why? I don't know. I'm going to create that config map from the file that I just made `heroes.txt`. And there's a new config map in town. `Dem heroes`. Notice `data one` means that there's one piece of data inside of it. Does that mean you can put multiple things inside of one config map? Yes, you can. Do people often do that? No, not really. Unless it's environment variables. If I were to run `cube control describe` against that particular config map, look at what we see here. Ah, the

contents of that config file are present just as text inside of the object itself. So you're not putting anything huge inside of config maps literally text files are the only thing big big things like database that's something about persistent storage and that's a whole different conversation. Now let's hop inside of our pod manifest from earlier. Let's not worry about resource control and all that stuff and let's



figure out how are we going to get that config map attached

to this pod. Step number one, we need to add that config map that we made as a volume to this pod. You really got to mind your P's and Q's when it comes to indentation with this, which is why it may be easier just to go to Kubernetes.io, find under the config map page a manifest that already has all this written down. So, survival tip for the exam because you could save yourself a lot of heartache that way. But what you'd see there and what I'm going to do here is I'm going to create a section

called volumes. That's right under the C and containers under spec because remember this is being added at the pod level. You can add as many volumes as you want. Give the volume whatever name you want as long as it is unique. Those were all DC characters from earlier, right? So let's name the volume DC Heroes. Although I really I could have called it anything. And what we're putting inside of that volume is a config map. Note the capital M. Boy, let me tell you how many times I've been burned by that. But camel

casing is definitely a thing in Kubernetes and it will bite you if you are not careful. Now, uh, the config map that we created, remember its name is dem heroes. So we're going to put dem heroes right there. So step one is complete. We've added the config map as a volume to this pod. Step number two, we need to add that volume to the container itself. So again, mind your indentation. I'm right here under the I in image along with everything else that I'm putting inside of that container.

I'm adding a section called volume mounts. And the dash remember means elements in a list. I could mount multiple volumes to this one container. I'm basically making a list of those. So first I need to put what is the name of the volume that I'm mounting. Remember how we called it DC heroes. So make sure that you are capitalizing correctly and doing all that correctly. Boom. Done. Next we need to specify all right where in this engine X container is this volume going to be mounted that is what

we set by the mount path and what's super cool about mount path is is that if you choose a directory that definitely does not exist heroes da does not exist inside of the engineext container as you can well imagine But with all that done, let's split our screen here and let's create this pod. All right, need to delete this pod here real quick. And let's make this pod as we see it here. Demo pod is created. Give it a second to come online. All right, I'm going to use cube control exec to hop

inside of that pod. And I want to show you right here. Aha, there's a new directory in town. Heroes. So, mount path clearly makes these directories if they don't exist. And if I hop inside of the heroes directory, here's where I want you to take a guess. What's going to be inside of that directory? Oh, the mount path's connected to the volume. The volume's connected to the config map. The config map contains a file. The name of that file is heroes. So, if we look, check it out.

There's all of our heroes. They're showing up inside of the container. And what's super nice is is that as long as this config map is up and running, I can delete and recreate this pod all the live long day, that heroes.txt file will always be there inside of that specific directory inside of my pod. Now, it ain't all sunshine and rainbows when it comes to volumes. Here is a very common mistake. Suppose that I want that file heroes.txt. I want to show I want that to show up inside of a specific

directory like inside of a directory that already exists. Let's put it in that Etsy engineext directory where that engineext.config file is located. Right? If I were to really quick delete this pod because I'm going to uh because I'm going to recreate it here in just a second. Cube control apply. Pod is created. But when I check get pods, uhoh wamp, we got an error. What you should be reaching for every time that there is an error, first thing is you're going to want to use your describe. But when we take a look

at the description here, it tells us we made the container, but it backed off, restart, it just wasn't successfully able to run. So cube control describe isn't really helping me here. It's not telling me what's wrong. But this will cube control logs is a cool command that we'll talk more about later that allows you to gather uh the standard output and the standard error that are generated by containers inside of your cluster. And when I look at this one container pod and the logs that it's got, it's saying,

"Hey, I tried to open engineX.config, but I couldn't. I couldn't find that file or directory. Do uh do you think it's a coincidence that that can't be found when that's the exact directory where we put our volume? No, of course not. Because uh here's what's happening. Mount paths will overwrite existing directories. And I know what you're saying because students say it to me all the time. Well, Chad, what if you were to put like uh heroes.txt at the end of mount path?

That would just create a directory called heroes.txt. That would work. Uh but uh probably not what you're looking to do because now your file is in a wrong directory. So, no, that won't work. It's in circumstances like this that you have to use what's called a sub path. And basically the way that I would explain a subpath is this. Let me type it out. There you go. Subpath means that instead of like mapping the entire contents of the entire config map to the entire directory at C EngineX, let's write a

subpath to just a file inside of the config map, which remember it's called heroes.txt. txt. That's what's inside of our config map. And we want to mount that file to heroes.txt inside of the Etsy engine X directory. So with that subpath now in place, let's delete our pod here. Let's recreate it and let's hop inside of it and let's see if it's working. First of all, it is. Looks good. Cube control exec. It's here somewhere. If I hop inside of uh Etsy engine X, you can now

see that our heroes.txt file is now peacefully coexisting alongside all the other files that are inside of this directory. So use subpath when you need to write to a directory where other files are present because otherwise it's just going to obliterate the whole thing. The natural progression from having learned about config maps is to start talking about secrets because secrets and config maps are very similar. They both take information and store it inside of themselves rather than storing it inside of the container.

The difference is is that secrets are well secrets. They are things like passwords, tokens, keys like generally sensitive information. Now, uh, secrets are created through an almost identical process to config maps. So, there's little to learn here, but there are certain warnings that I do want to give you about using secrets. Secrets on their own are not encrypted. They're stored in the API server inside of EDC, right? And anybody who has API access who can see that secret like inside of your cluster, they'd need cube

control to do it, but they'd be able to see it or change it or anything like that. Kubernetes does require

you to use your own encryption solution, which is not something that we're going to talk about. There's so many different ways to do it out there, but just so you know, secrets do not encrypt right out of the box. The other thing to know about secrets is that there are many different kinds. What we're seeing here on this table is that there's a couple that you can use for specific

circumstances like SSH authentication or if maybe it's like a username password situation with basic off. I'm going to be using that in a demonstration here in just a moment. These are all different kinds. The generic type of secret is one that's just opaque. It's meant to like be, hey, this is just some random like file or something and we don't want people to be able to see what's inside of it. All right, so here's a manifest for a change of how to create a secret.

And secrets are namespaced just like config maps and pods are. So make sure that your secrets are in the same name spaces as the pods that want to use them. Same is true for config maps. And what I'm going to do is that I'm just going to encrypt a secret uh password. Alta three is the value of the key password. And the type of secret that I'm using is basic off. Now you could like we were doing with a config map earlier use a cube control create command to do this. There's often you

know imperative ways of doing that. You would just have to spell out what type of secret is it and then hey we are putting in the uh environment well the call it environment variable put the keyword uh value uh from literal. So from literal is something you can use also when you're making config maps. Just makes it easy to put key value pairs inside of objects. But uh we won't be doing that. Let's just go ahead and uh create this secret from this manifest. And uh let's make sure that

it's saved otherwise that won't work. All right, there we go. So secrets can be listed out just like anything else uh that we make. But if I were to describe what is inside of that secret, my SQL secret, notice that it tells us that hey, there's a key password, but it doesn't show us what the value is. Remember, this is not secure. I absolutely could crawl inside of this particular object and see uh what that password value is. All right. So, you need further encryption for this to

work, but we can go ahead and take this to the next step. All right. What we're about to do is we are about to read in the contents of that secret so that it can be used inside of this MySQL uh database that we're putting together inside of this container. And everything you're about to see here could also be used with a config map. So, again, secrets config maps very interchangeable in how you can do things. Uh, one of the keywords you can set for a container is environment variables. So if there was

an environment variable called my SQL root password and there is the MySQL image has that environment variable already defined inside of it. You could just go ahead and say that the value is alt to three. Yeah, that'd be dumb. Yeah, let's just put our passwords right in plain text inside of our manifest, said nobody ever. My god, all of the security people just had a stroke. No, instead what we're going to do is we're going to say that, hey, the value of that environment variable is going to be taken from a

secret a secret key that is referred to by the following name. Now, do you remember what the name of our secret was? Cube control get secrets. Yeah, it was called my SQL secret. So that's that right there. And inside of that secret, remember we want alt to three to be the value for that variable. Right? So inside of this secret right here, we are looking for what key points to that value password. Right? So,

what I'm going to have to do down here on the left side is that I'm going to say that the key we're

looking for is literally called password inside of that secret. All right. So, uh with all that in place and if I were to save this and go ahead and create my pod at this point, pods created. Might take a second to get stood up. Whoop control get pods. All right. - W stands for wait. It means just give me updating output like as changes are made. We'll continue once this is up. Oh man, underestimated that. Okay, so it is up and running now, which goes to show containers don't always spin up

instantaneously. We're going to uh do a cube control exec inside of this container. Let's test whether or not this worked. I'm just going to for the sake of demonstration use the echo command on that environment variable, the one that we set right here, which just goes to show secrets ain't encrypted. You'll notice that right there. You can see that that's the password that we set inside of our secret. So, it definitely does work. Just make sure you got your ducks in a row when it comes to actually

setting up encryption inside of your setup. And for the exam, don't worry about encryption. that's not something that you'll be asked to set up. So, something super important to know for the sake of troubleshooting as well as just keeping your sanity is logs. And the way that it works is that every container inside of a pod generates its own logs. And the logs that are generated are always what the standard out and the standard error of a given container's output may be. To demonstrate this, take a look at this

manifest. We've got one pod that has two containers inside of it. Both of which are running busy box, which just means that we can on a basically infinite loop print out like the date as well as whatever the value of i is. And this one is pausing uh for one second every time that it prints. This one is pausing for 3 seconds every time that it prints. So, what we're bound to see is a difference in the output that's going to be created by both of these pods. Now, let's go ahead and create this pod here super

quick. And once that's up, we're going to explore this new cool cube control command. CubeControl logs. Now cube control logs you don't have to specify oh that's the logs of a pod or because that's the only thing that this command will look up the logs of pod containers. Now keeping that in mind if I were to check the logs of this pod which is called counter you got to ask yourself what sort of output am I going to get because there are two containers here. If I were to pull the trigger on

that, there you can see is the output that it's printing out the date and it's clearly happening every second. But if I scroll up a little bit, it says defaulted container count out of count and count by three. So this is just an example of the significance of the order in which your containers are listed. this container was listed first. Therefore, this container was built first. Therefore, this container is the one that by default we're checking the logs on it. But the cube control logs

command is pretty versatile. There's a lot of cool things that you can do with it. Uh for one thing, I can do dash c to specify I want to like get the logs specifically for this container. And you can see that these are in increments of 3 seconds. So this is definitely the second container. Show you the versatility of this just a little bit more that uh knowing all of your options that come with cube control logs can really help. If we were to do all containers, I will actually get all the logs from both containers being output

all at once. What you might also want is to get a stream going that these are being generated every 3 seconds. So if I put a dash f there on the end then you can see that this is updating in real time as new logs are being added and because we are doing these in uh increments of like a couple of seconds. What's super cool is is that you can also add in flags like this. Maybe all of my logs since 10 seconds ago. That's only just these three. So, there's a lot of cool things that you can do with logs, but let me show you

one other thing about the nature of logs and their impermanence. Because what happens is is that when a pod gets deleted, that pod is going to clean up after itself, which means all of the logs are going to get removed as well. What I'm seeing is is that this pod is on top of node one. This is just, you know, for your uh education and how logs work and the impermanence of them and why people use third-party logging solutions to kind of work around this. But I'm going to get on to node one

itself and I'm going to hop inside of the var log directory and check it out. There's a directory here called pods h. Let's hop on there. Sudo CD pods. What you complaining about? CD pods. All right. What's inside of pods? All right. What you're seeing here is that there is a directory called pods. It is root protected. So, let's do a little sudo here. And what you'll see inside of it is aha counter is the name of the pod which is in the default name space and these are all the other pods that must be

running on top of node two. So what that means is is that I could screw it. Enough of this messing around. All right. What I could do is I could hop inside of that directory here and show you. Look at that. There are two subdirectories, count and count by three. Those are the names of our manifests uh containers, right? So just in case you forgot, count and count by three are the names of those containers. And if we were to take a look inside of either of those, there's the log file. Now, if I were to delete that

pod, let's do a little watch and see how long it takes for this to delete. So, basically, every 60 seconds or so, depending on the cluster, Cublet will uh go and do its rounds, we'll do clean up and things like that. It's only a matter of time before this directory and therefore all the logs inside of it wind up getting deleted. There you go. Had my eye on the little clock there was about 45 seconds. So that's about right. So be careful that when you are cleaning up your pods, you're cleaning up all the logs that go

with it. There are third party logging tools out there. That's beyond the scope of what we're going to talk about. But now you know about cube control logs. Sorry for the visual jump scare, but I'm going to use my boine friend here to explain this next point. What we're going to be talking about next are labels. And I always thought the labels was a bad choice of term for what they are in Kubernetes. Tags will be much more appropriate. Uh I am from Hershey, Pennsylvania. And what you're seeing

here is a cow that very well could be at any of the dairy farms that are in my neighborhood. and cow are cattle. They are animals and they're being treated well, but they're just one of many inside of the herd, right? We put tags inside of ears so that we're able to track things like breed of like temperament of like farm or age. Like it's kind of like metadata that's being put onto these ear tags that helps us classify and be able to tell like one cow from the next because they're just

members of the herd, right? We don't typically keep cows as as pets. They're cattle. And I say that

respectively because that's exactly the concept of how things work in Kubernetes. We call them labels, these ear tags. And you can put them on any kind of object from pods all the way up to nodes themselves. And the purpose of labels is be able to group, view, and operate like a subsection of multiple objects all at once. And it's one of like the primary ways that Kubernetes objects are

identified. And labels are no big deal. They are ser and labels are no big deal. They're simply key value pairs. So if you take a look here under metadata, we have labels. There are one, two, three, four labels inside of this manifest. They must be key value pairs. They can be pretty much whatever you want. Uh the best labels are descriptive and unique. And you got to be careful as you're going to find out about putting the wrong label on the wrong object. that can lead to some uh definitely not good

unintended consequences. But it's really easy to add and remove these labels. Let's talk about that before we start using them in real practice. So let's take a look at creating some pods with labels and see how we can manipulate and use them. So let's make a pod. And as you can see from our manifest there, it's dead simple. And you'll also see in the manifest that there's no mention of any labels being put on this whatsoever. We're going to go ahead and create this

pod. And I want to show you that if I were to type `cubecontrol getpods d-show labels`, it'll give us the typical get pods output. But now there's also this column labels none. You can see we did not put any inside of the manifest. First thing we need to know is how to change, update, or remove labels on our pods and other objects. So, let's just get a little stream going so we can see this update in real time. There we go. First method is to go and edit the actual manifest itself. So, under

metadata, labels. Remember, labels are definitely always key value pairs. So, cool beans. That's what I want to do. If I do a cube control apply, that's a totally legal change. Kubernetes is happy to make changes to running pods. And as you can see up there, it's got the label cool beans on it. But there are other ways to add or even remove or update labels on top of pods, and those are using specific cube control commands. So, cube control label. I'm going to label that pod, which is called

demo pod. And let's throw another label on there. Let's say that I want the label awesome sauce. Be careful though, because when you're working with labels at the command line, you can't use colons like you would in a manifest. You got to use that equal sign. And that will add to the existing labels that are already there. Now there's no limit to the number of labels realistically that you can put on given objects. But if you try to put some other label where the first part the key

is something that's already a label on the object. So awesome training that's what this is. Ah I got this error that says that oh no that key awesome already has a value. So you can update labels if you want to change them. If I want awesome value to be training, then I would just add `d- overwrite` onto that. And you can see it changes it. You could also make this change in the manifest. As far as removing labels goes, uh, that's super simple. All you have to do is just take that command there, include

just the key, the first part, and put that minus sign on the end. So that's removing awesome and now I'm removing cool and now all the labels are gone. When you'll see later when we're talking about deployments and services and things like that, these are other types of objects that are highly dependent on labels. So being proficient at like manipulating them is super important. Now there's one

last thing I want to show you that is really neat about how to work with labels. I use this a lot

and I think it might help. Dashl, we're going to put this on our cube control get command. Dashl allows us to add a column where at the top we have the key. So let's say that the key is alt 3. Check that out. See how there's a column there called alt 3. Let's label this pod and let's add alt to three equals awesome. Notice that that's showing up. And if I were to create another pod, let's just call another pod. This one is using also the engine X image, then notice that it's blank.

So, in the occasion that you need to compare pods and like see which ones have a certain label and which ones don't, that can be a lot cleaner of an output. And it's something that I use in my label work. Oh, almost forgot. Now, uh the other thing about labels is is that it enables us to do things like trigger multiple objects all that have uh the same label on them. So, I'm going to create another pod here. And I'm going to label that pod called another pod. And let's put the label alt to

three equals awesome on top of that one as well. All right, we have three pods here that are totally not connected in the least. And when you are working inside of a like production sized cluster, you're going to be dealing with hundreds of pods, many many pods. How are you supposed to pick out from the haystack all the needles that you need? Well, that's what's nice about this. We can use selector that I want to only grab those pods that have the label alt3 equals awesome. See how it's only

returning those pods. That's important. And like I said before, tools like deployments and services use labels like this to kind of suss out which pods they're looking for. Let's talk about the brutal inevitability of what happens to pods. All pods die. They are not meant to last forever. And pods can die for any number of reasons, like due to fluctuations in traffic or errors or when you're trying to make updates. Point is, you should never expect pods to stick around. But what you probably

would want is that if a pod were to be taken down, like due to an error or like suppose one of your nodes goes down, you want it to come back, right? Well, if I were to delete one of the pods that you're seeing here, that pod's going to get deleted, but it ain't coming back, right? Definitely not bulletproof, one would say. which is why creating pods inside of production environments, pods are rarely ever created on their own. Instead, they are made as part of deployments. So, check this out. We're

going to create a deployment. We'll take a look at the manifest in a little bit. Let's call this demo deploy. And I want this to use engine X because it's a simple and easy one to build. And check that out. It made a pod named after the deployment. If I type cube control get deploy or deployment if you're feeling fancy. Notice that it's writing that out. It's also saying that one out of one is ready. Ah, how interesting thing is is that deployments are able to create multiple clones of the same pod

called replicas. and putting the cart before the horse a little bit here. I'm going to use a command cube control scale. There's other ways to do this, but I'm going to take that deployment called demo deploy. And I want the number of replicas to go from one to three. And look what's happening at the top of my screen here. All right, we got two more pods joining the battle. And what's going to happen is that this deployment is going to ultimately make sure that there are three pods, no more,

no less, constantly running inside of our cluster. And here's what's really cool. Delete pods all whack.

These pods are all getting deleted, including those two little demo pods earlier. But look what happened. The deployment replaced them. So, what you're seeing here already are two of the biggest advantages of deployments. The fact that it allows you to scale. I can add more of a pod or I could scale down. I could get less of a pod. Let's take it back down to one. And if any of my pods

should die, and they're not supposed to, the deployment is going to make sure that they come right back. Now, what can be tricky about deployments at first is their manifest. It's slightly longer than what we're used to seeing. so far in this course. But not to fear because what you're seeing here is a basic pod manifest, right? We've seen this plenty of times by this point. What a deployment does is that we take away certain parts of that pod manifest. For instance, we don't need to worry about

naming the pods. So that part of the metadata, we're just taking that right out. Remember, the pods are going to be named after the deployment that makes them. So, that's something to keep in mind. The new stuff is what you're seeing here up at the top. And let's break down exactly what it is that we're seeing. There are two metadata sections inside of this manifest. Don't get those confused. Uh, the one that you're seeing up here is the manifest for the deployment, and this is the manifest for

the pod. Remember, deployments make pods. That is their goal. So part of the deployment manifest will be the manifest for the pods that it's making. Right? All right. Now deployments can have labels too. Anything in Kubernetes can have labels, but it's not required. So don't worry about that too much. Spec. Again, this is spec for the deployment as opposed to spec for the pod, which is down at the bottom. And here you're seeing that when I was using cube control scale at the command

line to increase or decrease or set the number of replicas a deployment had. That's a number that you can set inside of the manifest itself. And you can totally edit the manifest and do a cube control apply. That will increase or decrease whatever the number of replicas to whatever that number may be. Now here's where things get really super important. Remember when we were talking about labels about how labels are used by many objects in Kubernetes to identify like to access other resources.

That is exactly what's happening right here. We have a deployment that's basically saying all right listen up all of the pods I will select them that match the following label app engine X. If you have that label, you belong to me. Engine X deployment. Robble robble. Robble. All right. So that's important. Okay. Our deployment is saying any pod that has that label on it belongs to that deployment. And what you'll notice is template is where we uh start describing, okay, what's the template

for the pods that are going to get created by this deployment. Notice that part of that template includes putting the label app engine X on any of these pods. That's required otherwise they won't even be recognized by the deployment itself. So let's build this deployment using this manifest and let's take another look at it at the command line. Okay, I'm going to go ahead and I am going to create the manifest that you see right here. And immediately you can see that there are one, two, three pods

spinning up because that's how many we said we wanted to have for replicas. I'm using this dashl uh part of my get pods here which is giving us this column app engine x. And remember we said that all of the deployment pods need to have the label app engine x and that is clearly what we are doing right here. Now, I'm going to show you a few things. First of all, just uh if I were to take the number of replicas



here and change it from three to two, making that change inside of the manifest, then running cube control

apply will cause the number of replicas to change. Same as when I was using cube control scale. The other thing that I want to show you using some of those cube control label commands we were doing earlier is this. I'm going to grab one of those deployments, the very first deployment. And notice that, of course, they got these random characters. That's to make sure that each pod has its own unique name. That's a dead giveaway that this is a pod made by a deployment or something similar. I'm going to take

that first pod and I'm going to take the label app engine X off of that pod. Now, what do you suppose is going to happen when I do that? Let's see and find out. Interesting. This is the pod where I removed the label. And look what the deployment did. It didn't go out and look for the pod that lost the label. That deployment assumed, oh, that pod's gone. I'm going to go ahead and create a brand new one to replace it. Because the rule is always maintain this number. No more, no less.

Now, what do you suppose is going to happen if I were to put that label back on top of that exact same pod? Well, like I said, it's two replicas, no more and no less. And uh it wound up keeping the original pod, which is nice. But try as you might, if you were to delete these pods, they're just going to keep coming back. Uh they are absolutely bulletproof. The only way to actually get rid of all of the pods that are inside of a deployment is to either delete the deployment itself or scale the number of pods down to zero.

So soon as I deleted the deployment, you notice that it's gone. Now, there's one last thing that I want to show you here just before we start doing some death by PowerPoint is I'm going to go ahead and I'm going to create this deployment one more time. What is technically inaccurate that everybody says is that deployments make pods. That's not right. The pods that are being created by this deployment are actually being created by what's known as a replica set. Replica sets are an

much older type of Kubernetes resource than deployments. They've been around since the very beginning. The problem with replica sets was is that trying to swap them out inside of production was getting to be a little bit too much of a headache. But to prove to you that this is the case, let me show you something here. If I were to show uh what pods we have, notice that the replica set is named after the deployment and it has its own unique little string. These pods here are actually being named

after the replica set. And you see how they're sharing the same value. So, it's kind of interesting and it's just a way that you can see which pods are being made by what replica set at a glance. All right, but let's talk a little bit more about some of the theory behind using deployments. All right. So, as you've already seen, deployments create replica sets, and replica sets are what create pods. And where those pods go on which node inside of your cluster, unless you are setting otherwise, the

scheduler is what's going to make that decision. As long as the number of replicas that the replica set is supposed to make exist, it's happy to let them live and let live wherever they may be. Now, why even use deployments if replica sets are doing all the work? Ah, now here's the rub. Deployments can do everything replica sets can do because deployments after all make replica sets. But what they can also do is that they can version control. And this is what I was saying earlier about how replica

sets weren't working so great inside of production. You can roll out and roll back changes that are being made to your pods. And you can do that in a way that there is zero downtime for your clients. They'll never even notice that the pods got swapped out. And unlike other types of objects where you make changes to the manifest, if you try to push those changes against a running resource, you'll get errors a lot of the time. But deployments, they expect change. They are designed to make change. And I'll

show you how that works in just a sec. All right. So, here's what winds up happening. You have a deployment that has that made a replica set that made a whole bunch of pods, but you decide that you want to make a fundamental change to how those pods are created. What winds up happening is that the deployment doesn't fix this replica set. No, no, no. It goes ahead and creates a new replica set, a second one. And as part of this rolling update process, which is the default, you'd have to like go an

extra step to make this not be the case. The way that it works is that it will surge ahead one pod. So now we have four pods instead of uh three. And then it will delete one of the old ones. Notice at no time do we have less than the normal number of pods, three. Then it surges ahead. It adds one new pod. Then it removes one of the old pots. Then it adds that third and final pod. And then it removes that last pot. So this is how deployments are able to make these changes without actually any downtime.

Sure, there might be a moment where you're kind of split between two different versions of a deployment. There's other ways to get around that, but if you're trying to roll out simple changes, deployments are designed to do that. Now, what's also super neat is that you can roll back those changes. Watch this. If I wanted to go back from this version back to this version, all we have to do is have the deployment go back to using this replica set. So, it's almost like this goes back in reverse. One of these

pods gets removed over there. new pod gets added over there, gets added there, gets added there, gets added there, gets added there, until eventually we're back to three pods here, and no pods over there. So, it's a really neat little system, and it's something you'll be using to create the majority of your pods in Kubernetes. Now, let's demonstrate that a little bit. I'm going to go ahead and I'm going to create the manifest that you see over on the left hand side of my screen. I am making 10

pods using this manifest. And you'll see what it did is that immediately it created a replica set. So what you're seeing here is the first and only replica set that this deployment has made so far. Now suppose that I'm making a change. I want each of those pods, all 10 of them, to not have one container inside of them, but two. And let's just say that oh I don't know we want to put like a reddus image inside of it. It doesn't really matter what the image is but the point is you

cannot add new containers to pods that are already running. So this will be a good demo. Now I want you to watch very carefully on what happens when I hit enter here. You're going to see a second replica set appear. you're going to see the number of available pods for that replica set go up and the number of pods for this replica set will go down because that's that rolling update that I was telling you about earlier. So there it goes. One is increasing, one is decreasing, effortlessly flowing from

one to the next. Boom. Done. So, all 10 of those pods have been recreated, and now all of those pods

have the two uh containers inside of them, as you can see right here. But here's what's super sexy about deployments is that you can not only roll out changes, you can roll back changes. So what I'm going to do is I'm going to use a whole new uh command here cube control rollout history of our deployment called demo deployment. This history keeps track of how many revisions has this gone

through. And there's lots of cool things you can do with this command that we'll talk about more in the full course, but I do want to show you just this one. If I were to do a cube control roll out history undo, what that will do is that it will take my current revision. I'm in the second revision right now. And it will roll it back to the previous revision. That's possible because all we're going to do is we're going to go from 10 down there to 10 up here. So, as soon as I run that command, oops, it's

cube control roll out undo on the deployment demo deploy. There we go. You can see that uh the numbers are switching and now it's rolling back one at a time just like it did the first time when we rolled it out. Next, we're going to talk about storage. Ooh, fun. Storage, Chad said sarcastically. Storage is complicated. Uh, not the least of which is because every company, every team out there wants to use storage differently inside of their cluster, like where the storage is located, what kind of storage it is.

Uh, it's just a big kurfuffle. And Kubernetes has gone through a lot of changes over the years and how it handles this. But the good news is is that no matter what type of storage you may use, whether it's AWS, as I'm just using as an example here, or something else, it's going to follow this exact same formula. So suppose that we uh need to put storage uh for our containers to access. Now, of course, putting storage inside of the container itself is no good because containers they die all the

time and containers are stateless and containers are supposed to be small. So, we're not putting gigs of data inside of containers after all. They need to be stored remotely like in the cloud or on a disk or something like that. So, what we're going to be doing here is that we need to basically chart a path between the container inside of our pod and our storage, which might be in the cloud or on a disk or wherever. Here's the process. First thing that you need to do is that you need to teach your cluster how to handle

the storage that you want to use. Kubernetes is storage agnostic, which means it's team AWS, it's team Azure, it's team whatever you want, baby, as long as you teach it what it is that you need it needs to know. That's done through creating something called a storage class. Once you have a storage class created and now that DNA is inside of your cluster, you are then able to start allocating, call them chunks of storage space that can be set aside and used specifically for your pods and their

containers. So let's say that out of my cloud here there's like a 3 gigabyte chunk that I want to uh access that is saved inside of something called a persistent volume. So persistent volumes are what teaches your cluster on how to create certain types of storage and handle it. Persistent volumes are what represent that storage itself. And the persistent volume AWS that you're seeing here will be made uh entirely differently if I was using Azure or something. It makes training very difficult because you know the

parameters are so different from vendor to vendor. All right. So that must be it right. No that's not it. All right. Everything in Kubernetes is very decoupled. All right. The persistent volume represents the

storage. What we need in next is something that represents access and that is called a persistent volume claim. So I suppose you could visualize this as the persistent volume claim is the key and the persistent volume is the locker. All right, these two are wedded together and it is a monogamous relationship. There is

one persistent volume claim that binds to one persistent volume. So there won't be multiple persistent volume claims mount mounting up to the same persistent volume. Doesn't work that way. But it is that persistent volume claim that gets mounted as a volume to the actual pod itself. And if you watch the section on config maps and secrets when we already walked through how volumes are added, then it's a very similar process. All right, you add the volume to the pod. You then add that volume attached to the

pod mounted to a directory inside of the container. And there you go. We have created a full flow of storage that anything that gets saved inside of this directory by the pods container. Let's work it back here. Oh, the mount path connected to the volume. The volume represents the persistent volume claim. The persistent volume claim is connected to the persistent volume which is abstracted a space that's taken from the storage space. The storage space is uh recognized by Kubernetes because you taught it through

a storage class and that's how you store your stuff. God. All right. Now it should be pointed out like much like when we were talking about config maps is that you can mount mult you can mount a config map to multiple pods you can mount storage to multiple containers because after all if both of these pods are say part of the same deployment they are identical to each other airgo they should have access to the same storage then attaching them to this persistent volume claim is uh perfectly doable and very easily

done. Now, in the demo that I'm about to do, we are going to create a persistent volume, a persistent volume claim, a pod that mounts it, and we're going to see data persist when we do it. All right, that being said, this is going to be a very low tech demonstration, although everything in it is absolutely fundamental, no matter what type of storage you might use. But we're not going to play favorites between Microsoft and you know all that stuff like uh it is what it is. So what we're

going to do is that we are going to use an extremely simple storage class called the manual storage class. What's that mean? It means that we are just going to manually choose a node. Node one, node two, I don't know. We're gonna pick a node and we are going to take a chunk of storage space from it and use that as our persistent volume. So that three gigs that's being represented by this persistent volume is going to be taken from one of our nodes. Perfectly valid way to do it if a little primitive.

We're then going to create a persistent volume claim that maps to it. Once we've done that, we're going to actually see them bind together. We're going to take the persistent volume claim. We are going to make sure that we write a manifest that mounts it. Now, what we're going to do when we make this pod is that we are going to intentionally create it on top of node one. So, you're also going to see how you can force pods to go onto certain nodes. All right. What's going to wind up happening is

that because this pod goes on node one, this persistent volume will take its storage from node one as well. And what's going to happen is that we're going to make a file over here inside of our mount path data. And what we're going to see is that that file is actually going to be physically located on node one.

So if you work all the way back, we should find index.html there. All right, let's go to the command line and let's try all this out. Okay, so we've got three manifests that we need

to make here. And we're going to put them all inside of the same document because why not? And first thing we're making is a persistent volume. And at first it's not so crazy. It's not super long. But we do see some new things here under spec. Storage class name is where you select. All right. What storage class am I using? Like in order to like build this persistent volume. You need to have created the storage class in advance of having done this. Most storage classes you can just find the manifests for them

like with whatever vendor it is that you're using. So that's good news. Capacity is how large is the persistent volume going to be. And because we are using just a local storage here manual host path means where on the node is going to be the directory where we're taking the three gigs of storage from. So in other words, anything that gets saved to this persistent volume, we should find on the node inside of that directory. Now, access modes, uh, there obviously are several different things

that you can set there, but read write once means that the volume can be mounted as read write by a single node at a time, which means that just one node can be writing to it at a time, right? It's the most common type of access for like a real basic type of storage like this. All right. Now, if I want to add uh something else inside of the same manifest, I can do that. YAML separates one document from the other by using those three dashes that you see there. What I'm now going to do is I'm going to

add my persistent volume claim. The only thing that's needed for this persistent volume claim to find that persistent volume is that they have the same storage class. There are other ways to guarantee that they connect, but that's not necessary in what we're doing right here. It's got the same access mode. We're going to use up all of the storage that that persistent volume has uh available to it. And that's all that it's going to take. Now, of course, there are far more complicated

persistent volumes, persistent volume claims that you can make, and we explore those more inside of our full course, but again, we're keeping it super simple for this fundamentals course that we're doing right here. All right, so last thing that we need to do is that we need to create a pod. And what we need to do is we need to walk through all that volume mounting stuff again, right? So, remember what's the first thing that you need to do? You first need to mount the object as a volume. So under volumes,

we're saying that hey, I want the persistent volume claim called engineX PVC. Remember that is the name of our persistent volume claim up here. I want that mounted to that pod as a volume. Let's refer to the volume as EngineXPV storage. All right. So we now have the volume added to the pod. We now need to add the volume to the container. So in my container here, I'm going to add volume mounts. Again, the name of the volume matches what's down here. And as I've done with config maps

in the past, I'm going to pick a directory inside of the container where I want that uh persistent volume claim to be accessed. So if you go to slashdata you are going to persistent volume claim and if you're going to persistent volume claim you're going to the persistent volume. If you're going to the persistent volume you are going to the node. Right? So anything that we put in this directory is going to get written to the node. Now, speaking of the node, there's one last thing that uh

we need to do here for this demonstration is we're going to use the node selector uh parameter. There are several ways to force pods to go onto certain uh nodes in your cluster. And really, you should only be doing this when you have to, right? There's some circumstances we can talk about later, but basically, hey, only go on top of those nodes where the host name is node one. Obviously, this is going to put this on top of node one. So, let me get all my ducks in a row here real quick. Yeah, I'm just going to um nudge

the uh node selector over a little bit because I definitely did not uh indent that properly. Whoops. Yeah, it's got to be lined up like under a spec like with the C in containers. Oopsy daisy. All right. So, the moment that we created that, take a look at what we saw appear that we I know it's kind of wrapped around my screen and that makes it difficult to see, but our persistent volume claim is bound and that has been bound to our persistent volume uh excuse me, the persistent volume has

been bound to that persistent volume claim. All right. And everything's bearing out exactly as we said that it would. What's also interesting is is that when I said in my persistent volume claim that I wanted the capacity to be one gig, it's two. Why' it go to two? Because the persistent volume was two. And it's like, what am I supposed to do with this extra gig? Just waste it? Nah. So, it's kind of interesting that it automatically scaled that up. All right. So, now that that's done, here's what

we're going to do next. We're going to cube control exec inside of that pod and we're going to take a peek inside of that data directory. First of all, the fact that there's a data directory at all means that our volume mount definitely did work. And what I'm going to do is I'm going to echo help. I'm trapped in a file and I can't get out. I'm going to put that inside of a file called proof.txt. Now, let's hop on out of that. If that were the case, and if this

did persist, there's a couple of things that we need to try out first. First of all, I'm going to kill all these pods because I had a deployment running at some point. All right, that uh pod that had the storage attached is now dead. I'm going to rebuild that pod again. I'm going to hop inside of that pod again and inside of the data directory I'm going to see is there a proof.txt file. Right. Yes, there is. So that right there is proof that this is persisting. We are using storage. What

I'm also going to show you is that that pod that I made is going to uh be on top of node one because we used we entered that inside of the selector, right? Node one. Let's SSH over into node one really quick. And let me get this manifest up there at the top so I can prove something to you here real quick. All right. SSH node one and let's go inside of the mount data directory. Remember how we said that that was the host path inside of our persistent volume? Well, when I look inside of this directory, lo and behold,

there's proof.txt. And when I got that out, help. I'm trapped in a file and I can't get out. Just goes to show that the data is persistent, that it is in a location other than the container itself. And while this is a very primitive form of storage, no matter what type of storage you use, it's all going to come down to this. Storage classes, persistent volumes, and persistent volume claims. Okay, the time has come for us to start charting a course into the murky waters of networking inside of Kubernetes. And

I promise we're getting there. But showing rather than telling is always my style. What I'm going to do here is I'm going to create a deployment. It's going to be using Engine X. I'm specifying that the port is port 80. And I do need to put the two dashes in front of that there because that's something that's going

to be important for later. Uh I also want there to be three pods made by this deployment. And there you go. Nice quick and dirty deployment. Now what I'm going to do

next is that I am going to create a service exposing this deployment. Now what is a service thing is that in Kubernetes for any uh pod to be communicated with what it basically needs is a great big neon sign like you'd see flashing inside of Las Vegas basically saying here it is here it is here's the pod that you're looking for. Why is that necessary? Well, let me show you something interesting. If I do a uh cube control get pods and let's get a few uh extra uh columns of information out

there. You can see that each and every one of these pods has its own unique IP address. Right now, if I were to delete one of those pods, let's just go ahead and delete the very first one that you see there. Now that's 247.34. It's on node two. What guarantee do you suppose there is that that pod when it gets rebuilt and it will will have that exact same IP address or even that'll be on that same node again. Like practically none. All right. And that's an entirely different IP

address. It's still on node two because theuler figured, oh, there's nothing on node two. But point is that IP addresses they change and as deployments like get scaled up and down and as pods get deleted and so on and so forth. IP addresses are a constantly moving target. All right. So this is kind of the nightmare we find ourselves in with Kubernetes. Fortunately, we got a really cool, really clever solution here. Now I'm going to do this using a cube control command. Although we certainly could write a manifest to do

this. I'm going to expose the deployment called demo deploy. And as a result of that command, what just got created was a new type of object called a service. There are three different kinds of services which we're going to talk about a little bit later, but the one that we just made and is most common is known as a cluster IP service. And notice that this service has its own IP address. All right. Now, why is that relevant? I'll show you in just a second. Now, let's get all those pods uh

up and visible one more time. Again, get pods output wide. And let's open up another pane here. do a watch uh cube control describe our service called uh demo deploy. Now what you're seeing inside of this description is uh well a couple of things but for one I can see the IP address 172.16.3.105 105 that's one but there are one two three other IP addresses here where do you suppose those are coming from you might have already guessed but those are the IP addresses of all these pods that you see

over here right each and every one of them is being matched up and what's super cool about services is is watch what happens when I scale the deployment demo deploy from three replicas down to just one. Two of those pods get deleted and look what happens to the service automatically. It just tracks the IP address of the one remaining pod. And if I scale that up to like 10, and the next thing you know, bip one, two, three, plus seven more, 10 IP addresses. So, as chaotic as pods getting created and

destroyed are, here's what's really nice. Between all those changing IP addresses, do you see the IP address that did not change, not even once that whole time? It's the IP address of the service. So, think of it this way. If the service always knows where the IP address of the pod is and pod IP addresses change all the time, if you know the IP address of the service, you're gold because the service is what will point you in the right direction. So that's the method behind the madness

and that's how services work. We're going to talk more about those in greater detail, but first we need to set some ground rules. All right, so what do I mean by that? Well, now that that deployment has been exposed by a service, it means that other pods inside of your cluster can actually talk to those uh pods. So to do that, I'm going to make this pod here. It's going to do all my talking for me. This uh pod called client is running by. And by is nice because it lets you execute

commands like curl. And uh we can see if we can use it to get network connectivity. So, I'm going to go ahead and I'm going to make that pod client. I'm going to cube control exec inside of that pod. And I do have the curl command uh inside of this. Never mind. I'm not going to use curl. I'm going to use wget. And what I'm going to do is that I am going to do this inside of my container. I'm going to type wget then the name of the service demo deploy. Now remember the service is what is pointing

to the pods. The pods and my deployment are all running engineix images. So when I say I want to download like whatever is at that service at that pod look what wound up happening. It downloaded index.html. What's that? It's the welcome to engineext page. How cool. We've just had podto pod communication. We downloaded the welcome page. And right now, every pod inside of our cluster is able to do that. They can access and talk to and work with the pods inside that deployment. It's peace

and love, man. Everybody loves everybody, man. That's the security nightmare. So, we need to control this. While yes, pods do need to communicate, we also need to set rules about which pods are allowed to talk to which pods. And to do that, we're going to create a new type of object called a network policy. And I'm not going to lie, network policies from all of the hundreds of students that I've worked with in Kubernetes, uh, is just the most nightmare of a manifest you'll ever see.

Look at this thing. Granted, not all network policies are this big and ugly, but if I can teach you how this network policy works, then there's no network policy out there that's going to phase you. All right, ready? Let's get down to it. So, network policies, much like resource quotas and limit ranges and other things, are assigned to name spaces. So you attach the network policy and what that means is is that this network policy is going to affect just those pods that are inside of this specific namespace. So

that's an important thing to point out right off the bat. You need to target the namespace of the pods you wish to control. Right? So that's step number one. Step number two, when we crack open the spec. Oh boy. All right. Basically, network policy if you boil it all down is really simple. It's all about controlling ingress and egress, which is to say traffic that is coming in and egress, which is traffic that is going out. You can configure network policies to uh control both or just one or the other. I always kind of

visualize a network policy as like a helicopter parent, you know, somebody who's like right on top of their kid like all the time. Like I don't want you talking to that Jimmy down the street. No, he's not good people. All right? Like I want you to only talk to that boy Melvin next door. Oh, he's a good kid. That's exactly how a network policy works. All right. So, policy types is where you list what things is this network policy going to control. Ingress, egress, both, just the one. You

get it? Pod selector is where we basically are saying what pods are we controlling using this network policy. So if the network policy is the helicopter parent, then who is the kid who's getting parented? And what we're doing here is we're using match labels just like we did with deployments. Like I said, labels



are so important in Kubernetes. And basically the way this works is that if you are in the default namespace and if you have the label RO DB, you are about to have your traffic coming in and

coming out controlled and certain criteria has to be met. All right, so let's go on down to the bottom part here. Here is where we're going to be setting up what are the rules that have to be followed in order for traffic to actually work. So for ingress here, we're making our first list of rules. We're saying that traffic has to come from one of the following sources. Now, you don't have to use all of these. In fact, most folks won't. But again, this is the longest and dirtiest example that

I could find. So, we'll just go ahead and use that IP block allows us to specify a cidr range of IP addresses like it's got to be that pod in this range otherwise we're not going to allow that pod to send ingress except and you can advertise exceptions if it's in like this range or this particular one right here that it's going to be uh not allowed. Pods can talk to each other across namespaces which is interesting. So even if pods are in different namespaces if one is uh you know exposed

by a service it's not safe other pods and other namespaces can access it. So namespace selector is where we can say that hey pods in the default namespace that have the label ro db you are only allowed to receive traffic from pods that are in this IP address range or pods that are from any namespace that has the label project my project. So again, this is uh going a label that's on the namespace itself. You can put labels on all kinds of different things. So in other words, must be from this namespace or must be

from this IP range or and this is the most common way honestly that we see these getting established pod selector from specific pods like it must be from pod with this label roll front end. So to go back through all that one more time, listen up. All pods in the default namespace that have the label RO DB, the following uh changes are about to go into effect. You may only receive incoming traffic from pods in this IP range or pods from that namespace or pods that have this particular role. Make sure that you're being very careful

with the rules of uh how network policy works because these are all or situations and you got to make sure you're really spelling out very particularly in a test environment that that's exactly what you mean and what you need. Uh what you can also do for ingress is that for ports you can specify all right pods that are being targeted by my helicopter parenting. You can only receive traffic into port 6379. You could also list other ports if you want. Unfortunately, egress is very much the exact same thing. It uses very

much the same language except now we're saying all right pods in default with the label RO DB. You are only allowed to send traffic to pods inside of that IP address range. And when you do send it, you're only allowed to send it to their port 5978. So, you can mix and match all of these in either section or you can choose to not include them at all, which is perfectly fine. All right, but make sure that you're being very precise with uh the rules you're putting in your network policy and test

test. So, very quickly before we move on, let's demonstrate actual network policies in practice. Now, there are no network policies in uh my system at all at the moment. I can uh inside of my client pod send a request to another pod, an engine pod. And right there you can see I downloaded that index.html file. But what I'm now going to do is that I'm going to apply this network policy that you see here. And what you're seeing, this is what's called a default deny network policy.

What's happening here is that all pods in the default namespace are about to be told where they are allowed to receive traffic from and where they are allowed to receive traffic to. Pod selector open close curly braces means all pods. So we're selecting all the pods in the default namespace. Now looking at that network policy, where are the rules that say what we are allowed to do? Because remember a network policy, if you aren't doing what the policy says is permitted, it's not permitted at

all. Oh shoot, we never actually gave any rules to like what uh exceptions there were. Which means that this is why this is a default deny. Watch when I run this wget command again. It's hanging. It's not going to connect. So, the network policy is kind of slapping our hands here, so to speak. And, uh, that's no good. We need to change that up a little bit. Now, uh, if we go back and we take a look at the manifest for my pod that is trying to access Engine X, it's got the label run

client on top of it. Basically, if this is what I want to do, if I want my EngineX pods to be able to accept traffic from pods that have the label app engine X, then here's what I would need to do. I would first need to see what are the labels that are on my uh deployment pods. Looks like they've all got uh app demo deploy as their label. So, let's go over here and for pod selector, we're going to change things up a little bit. Match labels app demo deploy. So now this network policy

is no longer applying to all of the pods inside of the default namespace, just those deployment pods. And I'm not too worried about egress. I'm worried about ingress traffic going into those engineX pods. And what I'm now going to do down here below is, and I'm just going to paste this in is that we are going to uh specify where is ingress allowed to come from. ingress to my EngineX deployment pods can only come from pods that have the label run client on them. So having made those changes and

running kubectl apply to my network policy. So the policy has been updated. My uh client pod using busy box is still up here and running. I'm going to run wget one more time and see if it works. So what we've successfully done is that we've been able to configure our network policy to allow this specific interaction. Although literally any other interaction that these uh deployment pods are trying to make would be prohibited. All right, we got a little death by PowerPoint coming your

way here, but hopefully it's going to clear up a few things. Now, we had made a service prior by using the kubectl expose command. This is what a service manifest looks like. It's not super uh terribly uh difficult. But again, notice that uh we're using selector just like we did with deployments, just like we were doing with um network policies to say which uh pods are being affected. So our selector is choosing this pod. Oh, excuse me. Our selector is choosing this pod right here. It's got that label. Now the

question is what are the meanings of these port numbers that you see right here. Well, the thing is that target port refers to the port that is on the target. And since we are using engine X inside of this pod, remember X's container port is 80. That's written in stone. That's not something that you can change. Port. This value right here is actually the port on the service itself. And what this is doing is that it is mapping a direct relationship to say that hey if you go to this service at

this IP address and you go to that port at that IP address 3 423 it'll take you to this port to that EngineX

container allowing you to access the application. Now the reason why that exists is let's expand this a little bit. Whoop. It is possible for pods to have more than one container inside of them. And if a service is going to redirect traffic to that pod, well, how am I supposed to be able to access the specific container that I want? Well, the same rule here still applies. If you

go to this service and you access it through that port, it is going to take you to port 80 which is going to connect you to the engine X container. But look what we can also do is that we can add a whole other port protocol here that if you go to say port 4444 that'll take you to port 8888 which is running some other application. So services can be configured to go to different ports even if they are entirely different containers inside of the same pods that they are exposing. So that's the difference between port

target port and just wait because it's ports of plenty over here. We're going to be talking about node ports. All right. In this animation, we are going to start talking about pod to pod communication and services in a visual way so that you can see how these tools work and interact, which hopefully is going to help you understand that Kubernetes does a lot of networking stuff under the hood. It's really nice to not have to worry about these things on a day-to-day basis, but you really do

need to understand this stuff in my opinion. So in the story that I'm telling you, super simple cluster, one node, there are two pods. They're on the same node as each other. And pod number one there is all like, "Oh, gee whiz, I'd really like to talk to pod number three." So how does that work? We know from what we've learned earlier that pod number three needs to be exposed by a service. Now let's say that that service is already created. But the question is how is that service located? How is

it accessed? So on and so forth. Here is where the great and mighty kube proxy is going to come into play. And kube proxy is a lot like kubelet. Kubernetes loves putting the word kube inside of everything. And there is a kube proxy located on every node inside of your cluster. And what kube proxy has access to is the IP table of every single node inside of your cluster. Which means that the kube proxy is perfectly able to look and see ah there's a service here called pod 3. The IP address uh that that service is

exposing is 10.1.2.1. That's the IP address of the pod. So in this case, the kube proxy's job is very easy. It checks the service. It sees that that service is pointing to just one pod here. And as a result, our pod is given the IP address and then routing is able to take place. Once the IP address of the pod is located, it's game over. All right, that's the end goal here. But remember that the headache that we face with Kubernetes is that we have dozens if not hundreds of pods with constantly

changing IP addresses. So we need some method on top of this madness. Now let's scale it out a little bit and let's get a slightly more difficult uh scenario. Let's say once again we've got a pod that really wants to talk to one of those pods inside of the zombie deployment. We just named it zombie because all of these pods are identical. They're all replicas, right? It doesn't matter which pod this pod talks to. Okay, so that's an interesting uh conundrum because once again, we

reach out to great and mighty Kube Proxy for uh connection advice. So, what Kube Proxy is going to do is that it's going to check its service table. It's going to find one called zombie and it's going to see that there are three IP addresses to choose from. The K proxy is responsible for setting up the rules on or rather obeying the rules of which IP address gets selected. And what will wind up happening is that it

will use a roundrobin by default approach that hey if the last time somebody came knocking on my door asking

for a zombie pod and I sent them to 1.3.1 this time I'm going to send them to 1.3.2. So it's cube proxy that's ultimately making the decision here which one of these three pods is going to be the one that winds up talking to busy box. And what also happens as like a mwah part of that process is that proxy will also make sure that that pod is healthy before a decision gets made. So if this con pod is still in container creating or it's erroring out then that pod won't get chosen. It'll instead hop

down to the next one and then it will choose that pod instead. So in this story we'll say hey I'm going to go with node two. All right that's just how the cookie crumbles in this story. And now we're going to take it even further back. Once again, let's say that we've got a deployment that's making pods. The deployment's making three identical pods. Two of those pods happen to be on node two and one of those pods happens to be on node three. That's just the way

that it works out. And now that we're looking at a multi-node cluster, everything that I was telling you about cube proxy applies to all of the nodes. Now, take a look at the table that I've got in each one of those nodes that you see here. What do you notice about it? They're all the same, right? So, it doesn't matter where you go. any cube proxy in any node whether you have a cluster of one node or 50 nodes they're all going to have access to the same routing information. So this uh begs the

question how is one going to connect to one of those pods? Well before I move on to the next point let's do a little uh exercise here real quick. Suppose that I have a pod down here on node three. This is using a busy box image. And let's say that it needs to talk to one of the zombie pods. And it says, "Oh, great and mighty Cube proxy. Please pick a pod for me." Cube Proxy goes and it looks at all these IP addresses. Is it a guarantee that Cuberproxy is going to choose 10.1.2.1? No, not at all. That is not

important. it can't be important. Cube Proxy will use roundrobin to determine what the next pod is going to be chosen. So, my pod that I drew right here might wind up talking to a pod over on top of this other node. And that's okay because think of it, if that pod had been created over here on top of node number one, and I said, "Hey, I needed to talk to one of those zombie pods." I'm gonna have to wind up talking to a pod on a different node anyway. So because things are always so chaotic,

we can't be taking the time to think, oh, is this pod in the same node as this other pod? That doesn't matter. All right. Now, everything that we've just been describing, pod to pod communication, has all been a specific type of service known as a cluster IP service. And there are three types of services. There's the cluster IP service, there's a node port service, and there is a load balancer service. And each of these services build on the one that came before them. So if we have

an idea of how cluster IP services work, let's take a step up now to a node port service. Now, uh what you'll see here is that our tables have changed somewhat. are zombie services are no longer cluster IP services they are node port services which means they have a node port this will always be a number between 30,000 and 32,000 something something something right point is node ports are always very recognizable all right nodeport services are uh the most primitive type of service that actually allows external

access into the cluster without using kube control. Check this out. If you have a client who has the IP addresses of one of your nodes and has the node port of one of your services, that client is able to access whatever pod is being pointed to. So, let's walk that through. If I were to send a curl to 192.168.0.12, 0.12 that is this node right here. When we reach that node, we are accessing it via port 31234, which is the node port that's attached to our zombie service. At which point, guess what? The

kube proxy says, "Ah, I see you're trying to access zombie." And then the exact same behavior from before occurs. Qroxy is going to use round robin to determine which one of these pods should we go to next. Is it guaranteed going to be this pod? No, we've already said that. It's going to use round robin to choose. So if it turns out 10.1.3.2 is the IP address that gets chosen. Even though we sent a curl request to node 3, we wind up accessing a pod on node two, which is perfectly

normal. And if that seems strange to you, well, imagine if we had a 50 node cluster and there's one pod on those 50. You have a one in 50 chance of guessing the correct node. That couldn't possibly work. So that's how node balancers work. Uh, excuse me, that's how node ports work in a nutshell. and it's a method of how you can access your pods ultimately from an external connection. Now the third type of service that we're going to talk about are load balancers and load balancers are more

sophisticated. Load balancers still use node ports as you can see here. But a load balancer requires some sort of external machine of to your cluster with an outwardfacing IP address to accept incoming traffic. If your Kubernetes cluster is in the cloud, ah they be they those folks Azure and uh Amazon, they love to take your money and they'll spin up a load balancer for you. It's coming from a place of jealousy because like we all do all our work on bare metal. But point is is that you do

need to provision that. Uh Kubernetes does not come with load balancers. It's happy to work with load balancers that you will provide it but that's not something that is included. But once you have a load balancer, what's nice about this is is that your client can now go to that load balancer instead of going straight to one of your nodes inside of the cluster. And what the load balancer will do is that it will say, "All right, enough of this roundrobin stuff, which is not true load balancing. I'm going to make a uh

educated decision about which pod is going to wind up getting selected." And thanks to uh the fact that load balancers also use node ports, it can choose whichever node it wants. It can access the IP table through the node port. It can choose the IP address of the pod that it wants and can ultimately chart uh routing that way. All right, it's pictures worth a thousand words. Let's get down to the command line and let's actually make some services and let's see how node ports work. So, I've

created this pod here. And before I start setting up any services at all, I'm going to go ahead and open up a port forward. And let's just do port 2224. And that is going to port 80. And so long as that port forward is open, I can curl localhost. And uh the way that this is set up is that inside of this pod there is a endpoint which is uh called slashstatic and that's just a little repository that has a file in it called engineext.txt. Inside of that little txt file says you done did a

networking. So notice I needed a port forward in order for this to work. like I needed kube control access in order to access what was going on inside of that container. All right, but we're going to change that

because what we're going to do next is that we are going to create a uh we are going to expose excuse me rather that uh pod with a node port type service. So, we're going to do cube control exposed pod engine x configured. And what I need to explicitly state here is that our type is node port. By

default, it's going to be cluster IP. Otherwise, so I'm going to go ahead and expose that pod that created the service. And if I were to describe the service that I just created for EngineX uh configured, it's pretty much the same as what we'd seen before. But notice that the type says node port and that there is also now a field node port 30337 that was assigned automatically. So that number right there 30337 is something that uh we need to remember. Now what I'm going to do is

this. Let's get that curl command back in here. What I'm going to do is that I'm not going to through my local hosts uh cube control connection. I'm instead going to go to one of my nodes. So let's do it this way. cube control get nodes outputwide. The IP address of my first node is 10.14.24.245. Let's do this. I'm going to curl. That's node number one. I'm going to go to that node port 30337. That should access my uh service which should access the pod. Which means that if I throw

slashstatic.txt txt onto the end of it. You done did a networking. Super cool. This is just a straight curl command. I'm not using cube control or any other kind of special permissions. I just had the IP address of the node and the node port. Now remember what I was telling you earlier that it does not matter what node the pod is on. As long as you are using the IP address of one of those nodes, uh, it'll work because cube proxy is everywhere. So to prove it, I'm going to grab the IP address of my second node

right here. And let's run this again. This is the IP address for my second node. Same node port, same everything else. still works, which just again is just kind of the proof that node ports are accessible and can be used to track pods no matter where you are in the cluster. And if you are planning on taking the CKAD or the CKA exam, it's pretty much a guarantee that there will be some node port service related question on that exam. And there you have it. That's our crash course in

Kubernetes fundamentals. But there is still so much more to learn. Though we've covered everything in this course from YAML all the way up to networking, there are still many more things that need to be learned in order for you to especially pass the CKA or the CKD exam. That's why I recommend that if you truly wish to master Kubernetes, get your hands dirty, and do what you need to do to succeed, check out Alta 3 Academy. All this content and way more is waiting for you, including a work environment,

the same environment that I was using in this demo. An AI instructor that will answer any question, no matter how simple or complicated, plus full mock exams for the CKA and the CKD, so you can test your skills and make sure that you are really learning and prepared to get out there and take Kubernetes by the horns. So, click that link down in the description and take the next step towards becoming a true Kubernetes expert. I hope to see you there. Make sure that you like and subscribe and I'll see you next time.